ScalaLex - Generating Lexical Analysis Tools in Scala

BSc. Computer Science

School of Informatics and Engineering, University of Sussex

candidate no. 181313

Project Supervisor: Dr Ian Mackie

25th May 2020

Statement of originality

This report is submitted as part requirement for the degree of BSc Computer Science at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years.

Abstract

In this project I have researched meta-programming with specific focus on the subject of front-end compilers. During the project I produced a program called ScalaLex. ScalaLex is a tool to assist in the writing of Lexer modules, compiler components intended to perform the tasks of scanning and screening of an input file into a list of token representations. ScalaLex takes a list of language definitions in the form of pairs of regular expressions and code actions and produces a scala program as output. ScalaLex is based on the design of flex, a generator that works in a similar way and produces C code as the output.

Acknowledgements

Thanks to Dr Martin Berger for supervising and providing ideas during the first half of this project.

Contents

1 Introduction **6**
1.1 Problem area . . . 6
1.2 Motivation . . . 9
1.3 User Needs . . . 10
1.4 Aims and Objectives . . . 10
1.5 Requirements . . . 11
1.6 Extension tasks . . . 11
1.7 Project relevance . . . 12

2 Professional and ethical considerations **13**
2.1 BCS code of conduct . . . 13
2.2 Ethical considerations . . . 13
2.3 Software engineering practices . . . 13

3 Related work **15**
3.1 Existing tools . . . 15
3.2 Regular languages . . . 15
3.3 Parser Combinators . . . 16
3.4 Language Modifiers . . . 16

4 Project Plan **17**
4.1 Tasks to be completed . . . 17
4.2 Task interdependence . . . 18

5 Implementation **20**
5.1 Processing of Regular Expressions . . . 20
5.1.1 Pipeline Overview . . . 20
5.1.2 Lexing of Regular Expressions . . . 21
5.1.3 Thompson Constructions . . . 22
5.1.4 Special case operators . . . 24
5.2 DFA Conversion . . . 25
5.3 Lexing of Input language . . . 25
5.4 Creation of Output program . . . 27
5.5 Programming techniques . . . 27
5.5.1 Functional Programming . . . 28
5.5.2 Recursive optimisation . . . 28
5.5.3 Object Oriented Programming . . . 29
5.5.4 Serialization . . . 29

5.6 Limitations of ScalaLex 30
5.6.1 Issues encountered during development 30

6 Other applications **31**
6.1 LaTeXmarkup 31
6.2 Emails 31
6.3 Text adventure games 31

7 Future work **32**
7.1 Planned features 32
7.2 Other planned projects 33

8 Conclusions and Reflection **35**

A ScalaLex Readme **38**
A.1 ScalaLex 38
A.1.1 Requirements 38
A.1.2 Instructions for use 38
A.1.3 Input Language 39
A.1.4 Built-in methods 40
A.2 Regular expression support 41
A.2.1 Regex Operators 41

B Tools and Abbreviations **43**

C Language Specification **48**

D Supervisor meetings **50**

E Original Project Proposal **52**

Chapter 1

Introduction

In this project I have done research into tools and techniques for generating front-end compiler components that allow developers to produce compilers for new languages quickly. Compilers are programs that bridge the semantic gap between a programming language and the hardware instruction set, or machine language used by every computer system. This gap arises because programming languages are abstracted away from the hardware to varying degrees to allow for long term maintenance, platform independence, and user-friendliness, however each piece of hardware uses a different instruction set, and some use different versions of the same instruction set, so maintaining code written in machine languages quickly becomes a Sisyphean task when trying to produce code for more than one computer system.

The development of reliable and efficient compilers is a costly and time-consuming process for any organisation. As such, many smaller organisations and hobbyists struggle to develop compilers intended for use in new languages which makes the development of new languages and the optimization of compilers a difficult task with a high barrier to entry. Compiler generation tools can help alleviate this by reducing the work required to create a compiler pipeline through the automated generation of compiler modules. Compiler generation tools are meta-programming tools that produce a new source program intended for use as a compiler. They target different sections of the compiler pipeline and produce more or less complete programs depending on how the generator is written, configured and used by compiler developers. Front-end compilation is best suited to producing compiler generators because it is the most abstract, most generalisable and most platform-independent component of the platform that only deals with the language syntax, structure and semantics.

1.1 Problem area

The evaluation of programs is in many ways an integral part of computer science, as it allows programmers to be more confident in their reasoning about how the programs will be executed. One of the most important tools in this process is the compiler, which allows programmers to ensure that their programs conform to a specific language, which gives certain guarantees about the programs depending on the language. Building compilers is a very difficult task because it requires a

deep understanding of several topics relating to how a high-level programming language can be converted into a machine instruction set. Whilst it is possible to directly take a program from an unformatted string into full program code and execute it in one step, it is:

- Difficult to implement for non trivial programs that have deeply nested program structures
- Likely to be inefficient at producing code
- Harder to optimise
- Highly prone to bugs
- Difficult to debug

As a result, the conventionally accepted method has become a divide and conquer approach, where compilers will produce several intermediate representations before producing any code in the target language. The most important phase is the parsing phase that produces an Abstract Syntax Tree representation of the program that captures its syntactic structure and encodes information about each coding element. Parsing is usually preceded by a lexing phase because it is difficult to create an AST without first lexing the string into a tokenised form. Parsing from raw strings is known as *scannerless parsing*.

Lexing

Lexing is a combination of the tasks of scanning and screening [17], that is,

1. Reading the string of text and separating the language symbols along a designated separator character.
2. Removing redundant information from the string, such as the separator symbols that do not convey information to be used later in the pipeline.

A lexer defines and enforces lexical constraints of a language, with a set of valid keywords. This is the first task in a compiler pipeline, as it extracts the semantics from an unstructured stream of text. Most lexers also build a representation of the code using a list of tokens[1] which contain information that is usually used for parsing.

When the lexer is run on an input text, the text is searched using a list of lexical rules that take the form of a regular expression followed by a token and/or a list of code actions. When a pattern is matched, that section of the input string is consumed, and the tokens are emitted to the final output. If any part of the input string doesnt match any rules, the entire string is rejected by the lexer.

Traditionally, lexers are implemented by using a table (2D array) variant of a Deterministic Finite state Automaton, with each correct symbol having another position in the table associated with it [1].

Alternatives to Lexing

Lexing and parsing can be combined into a single unit known as a combinator parser, which instead of passing the task of lexing to a separate program, uses the input string directly with a context free grammar to validate and create tokens at the same time as evaluating the structure of the Abstract Syntax Tree. This usually comes at the cost of efficiency, as pattern matching once per program against a regular expression is faster than matching against the context free grammar each time a new token is called. Scala can implement Parser combinators natively as a language feature. Scala includes a native parser combinator library that performs this function, and several others have been developed such as fastparse[18].

Automation

Compiler pipelines must be both reliable (guaranteed even) and efficient. This is because you would expect a typical program to be compiled many times over its life cycle and you should be able to accurately reproduce the result of every compilation for each given program. Because lexers use an arbitrary string as input, there is a very large input domain for the program, and so there are absolutely no guarantees that the input string will be well formed at this stage, as there is no encoded information about the program. This means that this compiler module can be vulnerable to errors; because programs typically vary a lot in function, complexity and coherence across a given programming language, the lexer will likely need to be run on a very wide variety of complex inputs. As a result, debugging and optimizing a lexer can be difficult, time consuming, and prone to error if done manually. In this regard, being able to utilise a tool to generate your lexer saves you lots of time in the design, writing and debugging phases of development because the generator will give you certain guarantees about how your lexer will perform.

Lexers tend to be written predominantly using the same programming technique, that is, pattern matching, meaning that writing a lexer is very formulaic and conforms well to automation. There is a very popular family of tools based on Lex[2] that produce lexers for a target language creating the main pattern match component automatically, usually written in C.

Automatic generation of lexers and parsers is very popular because lexers and parsers are part of front-end compilation and as such are hardware independent and can be used in many different compiler pipelines. By using a standardised language for writing lexers and parsers as defined by tools such as Lex and YACC, compiler engineers can focus their development time on code generation and optimisation steps, which can have a huge impact on the overall performance of a compiler pipeline.

Programs such a lex and yacc come under the category of meta-programs, that is, programs that use other programs as data. The simplest form of programs-as-data representation is a string representation, however this representation does not provide much utility or safety for the programmer because there is no tokenised information built into a string. A much more common approach to metaprogramming is to produce a tree representation or nested list representation of the program that contains syntactic tokens, allowing for more complex behaviour such as in the case of interpreters or reflection based

programming[8].

$$\text{Source} \longrightarrow \text{Lex} \longrightarrow \text{yylex}$$

$$\text{input} \longrightarrow \text{yylex} \longrightarrow \text{output}$$

Figure 1.1: The Input and Output for Lex, a lexer generator [14]

Some standardisation also extends to back-end code generation with intermediate representation target languages such as LLVM [12] that allows for more portable/universal implementations to be achievable and reduces the amount of work required to produce assembly for a given platform.

1.2 Motivation

Changing Environments and changing needs It is reasonable to question why should we produce new languages and new compilation tools for existing languages when the existing technological ecosystem for developers is so rich and so well developed. Why should we think about producing new languages when there are multiple options available for every possible scenario? The simple answer is that we do not know that we have fully explored all options, and in terms of optimisation of compilers and different compilation methods, this is still an open research topic with several different answers as to what is the best option. We also do not know what future challenges will exist that require new approaches to language syntax and semantics, because much of the demand for languages is intertwined with the needs of organisations within the software market such as Microsoft, Apple or Google, as well as new developments in platforms and how developers and users interact with computing systems.

One such example is the language *objective-c*, which was adopted by Apple as part of Mac OS X in 1996 after acquiring the company NeXT which had a license to use it. In 2014 Apple released their own language *Swift*[10] to replace objective-c which iterated on it whilst also providing full compatibility with objective-c libraries, effectively wiping out the demand for objective-c.

In the example of objective-c, there is a clear example of the business environment affecting the language needs of development, because the Apple technology community was pushed by Apple into adopting a new language for development of future applications. But technological needs can also necessitate new trends and innovations in the use of programming languages. This can be seen in the adoption of *Go* or *Golang* in response to the rise of cloud computing services.

Historically, some of the earliest programming languages were known as batch programming languages that were used in batch processing systems. These early computing systems are designed to run a program from start to finish without any kind of asynchronous execution, meaning that the computer runs the program and then produces a result, often as a paper readout. These batch languages were phased out with the rise of the personal computer, where asynchronous execution was a common occurrence within an event driven operating system.

As a result of these and other changes in the technology world, new languages and new compilers for old languages were produced to satisfy demands from users and from developers and organisations. Eventually, common practices were developed for producing correct and efficient compilers. One of the biggest contributions to this field was the 'dragon book'[1], which described techniques that were previously considered complex and difficult to replicate.

Existing tools for generating compilers Most popular compiler generation tools output a small set of well-known languages such as C, C++, C#, Java, Python etc. [7] but few of these generators produce code for functional languages, and of those, most output Haskell code. Very few lexer or parser generators produce code for Scala, mainly because Scala has built-in support for Java programs, however, these Java compilers do not take advantage of the many language features that Scala has to offer such as native pattern-matching and anonymous functions [15].

Scala also offers a very quick and easy method of producing combinator parsers from context free grammars that allows for rapid construction of lexing and parsing programs, however combinator parsers have a reputation for being slower than a pipelined lexer-parser combination. Parser combinators also do not support the same level of screening capabilities as a lexer written manually or produced by Lex.

I chose this project because I want to extend my knowledge of compilers, programming languages, functional programming and software engineering techniques, whilst also providing a tool that could be used by compiler engineers in the future.

1.3 User Needs

The intended users for this program will be compiler engineers producing General Purpose Languages and developers looking to create Domain Specific Languages. GPLs are langauges designed to be turing complete whereas DSLs are languages designed to be used for a specific task or for interfacing with a specific application such as a web browser (HTML).

Compiler engineers need to know that the lexer generator will give them a compiler that correctly implements and enforces their language syntax. Speed is also a significant issue for any compilation task because compilation is a ubiquitous task required for every program written in a high-level programming language. The developers also need to know how to implement their language in the input specification, so the language syntax for lexical rules needs to be made available in the documentation.

1.4 Aims and Objectives

The aim of this project is to produce front-end compiler tools that allow compiler designers to produce lexers and parsers for new languages quickly and efficiently. This will come in the form of a lexer generator that, given a regular expression as input, produces a lexer as output that will specify whether a given source program in the form of a string of text will produce a valid list of lexical tokens,

as specified by the original language. The lexer that will be produced will be implemented as a Scala program.

1.5 Requirements

F1	The generator shall implement an input language in the form of a context free grammar.
F2	When given an input language in the form of a set of rules, the generator shall produce a lexer that corresponds to those rules.
F3	The generator shall produce correct Scala code as an output.
F4	The lexer produced shall emit a list of tokens that represent the input string.
F5	The lexer produced should support code actions as well as outputting tokens.

Table 1.1: Functional requirements of the lexer generator

NF1	The generator shall be written in Scala.
NF2	The generator shall use functional programming techniques.
NF3	The generator should produce efficient lexers.
NF4	The generator should be easy to use

Table 1.2: Non-functional requirements of the lexer generator

1.6 Extension tasks

Once the generator is complete, it would be useful to create a fuzz tester that could be used for more rigorous verification of the lexers that are produced. Fuzz testing is the process of constructing test data by adding noise to correct inputs generated grammatically. This can also be done as Whitebox testing [11] where actual program runs are recorded and then mutated to create new test data. This fuzz tester could also be integrated into the pipeline for the lexer generator to allow for instant verification of the output.

As an extension task, I would be attempting to produce a parser generator, which, given a context free grammar will produce a parser that will check a list of lexical tokens against that context free grammar to ensure that it produces a valid abstract syntax tree to represent a program structure.

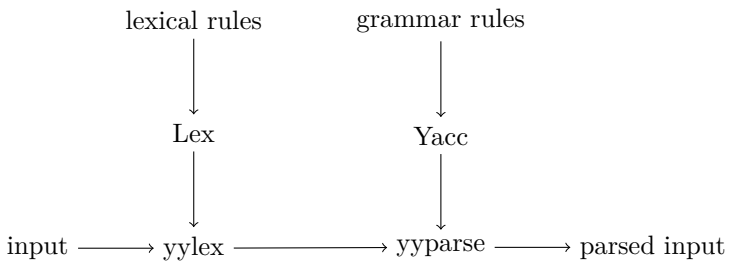

Figure 1.2: A front-end compiler pipeline structure using Lex and YACC [14]

As with the lexer generator, fuzz testing could be used to verify the outputted parsers. Additional future work could be done on comparing the efficiencies of program evaluation tools such as the lexers produced by this generator, and the combinator parsers in Scala.

1.7 Project relevance

This project is relevant to several areas within my field and my degree and will encompass techniques from several modules including compiler design, software engineering, programming concepts, comparative programming and program analysis.

Compiler design techniques are necessary in producing the lexers and decisions will need to be made about how the lexer will be constructed and how the rules will be implemented into the output lexer.

This is a complex software development task for which the validity of the final product needs to be verified using software engineering tasks to ensure delivery. The software engineering techniques I will be using are detailed in the Software Engineering section of this report.

Techniques from the other modules mentioned will be used in the design and implementation of the DFA and the CFG meta-language.

Producing a lexer generator in Scala will benefit developers interested in testing the limits of the Scala language and its associated technologies such as sbt.

Chapter 2

Professional and ethical considerations

2.1 BCS code of conduct

During my project I have followed the British Computer Society code of conduct by:

- Seeking honest criticisms of my project from my supervisor and others
- Developed my knowledge and skills around the field of program evaluation
- Carried out my project with care and diligence by using software engineering techniques to ensure quality control
- Ensured that my report has academic integrity, citing sources and providing credit to third party content correctly
- Not misrepresented the nature of the produced code in my final report.
- Fulfilled any other professional considerations and BCS codes of conduct that may be relevant to this project

Also, the ScalaLex project complies with the BCS code of conduct by:

- Providing tools for developing new languages and compilers to make the field more accesible to developers.

2.2 Ethical considerations

There are no significant ethical concerns for this project, as it is a pure software development task involving no human participants.

2.3 Software engineering practices

To ensure the quality of my code, I have made sure to use several software engineering practices that help to very the functionality of the program.

Testing Unit testing is facilitated through the scalatest library. I have produced numerous tests of various edge-cases and combinations of regex features for the regex parser, and features of the core program are tested against an input file that includes examples of each type of input.

Once I have a working implementation of the lexer generator, I will write some input cases to attempt to catch mistakes on edge case rules, such as the difference between `'.*'` and `'[^'\n]*'` Where the first pattern matches a pair of quotes that are furthest apart and depending on implementation this may look ahead to the end of the line, like in the case of Lex, or, the end of file if \n or newline characters are matched as well. The second pattern, however will match the closest pair of quotes. For example, in the line
`'first' quoted string here, 'second' here`
The first pattern would match
`'first' quoted string here, 'second`
Because of longest prefix matching, whereas the second would match `'first'` correctly [14].

Logging Logging and trace of execution is facilitated through the scala-logging facade library in conjunction with Logback. Efficient and effective logging practices were essential for this project in debugging both the regex parsing module and the input lexing module, because these areas can produce errors that originate from different sections of the code, making it very difficult to trace without proper logging.

Version control In order to ensure that my submitted code is of a higher standard, I am using git in order to maintain a canonical order of contributions for my work. Using git has the advantage of support for many tools to manage workflow, such as git hook scripts that can run tests during commits and enforce constraints on the validity of commits based on their results [3].

Automation I have used a git workflow script in order to automate testing. This simple script builds the project and runs all of the enabled tests in SBT each time code changes are pushed to the repository.

Chapter 3

Related work

3.1 Existing tools

Lexical analysis generators There are several lexer and parser generation tools available already such as Lex [14] [1], FLEX, Ragel and re2c (see B.1), which can be used for lexer generation. The majority of these use a direct implementation of a DFA. As is conventional in lexing, Lex uses longest prefix matching, followed by the order of rules to decide which regular expression rule to match against.

Syntactic analysis generators ANTLR, YACC or Bison which can be used for parser generation. Parser generators may also generate and include a lexer in their result, such as ANTLR and YACC++. Some also include a lexer generator in the main software, whereas others use the APIs for popular lexer generators such as FLEX. Most parser generators use YACC or EBNF as a standardised language definition to allow for interoperability.

Both lexer and parser generators allow for code snippets, known as actions, to be written into their pattern matching rules and executed each time that pattern has been matched. This means that you can define manipulations to be performed on the consumed input string. A common action to perform is removing tabs (\t) and new lines (\n) and replacing them with spaces, as in most languages these symbols are equivalent separator symbols with no semantic value [1].

The topic of generating lexers and parsers is well explored within the industry, and many lexer and parser generators exist that produce fast and reliable lexer generators, most of which being written in an imperative style in C, or a similar language.

3.2 Regular languages

Lexical analysis uses some implementation of a regular langauge[5] in order to specify rules for each lexical token, that is, a given 'word' of the language.

3.3 Parser Combinators

Parser combinators are compiler components that perform lexical and syntactic analysis at the same time. Parser combinators are included in Scala libraries [15]

3.4 Language Modifiers

Macro processing Macro processors are special case compilers that convert a shorthand name into a longer construct[1]. These are usually very simple and can be created using only a lexical analysis phase. Macros operate in a similar way to function definitions in modern programming languages although functions are dealt with at a semantic level whereas macros are compiled away at the lexical/syntactic level.

Language extenders Language extenders are macro processors that allow for more complex program structures to be written easily into an existing programming language to add capabilities to the language[1]. This type of technique can also allow for more specialised or sophisticated programming languages to written on an existing platform, such as in the case of Java with Scala[15] or the C, C++ and Python family.

Assemblers Some compilers produce complete assembly for a specific target platform, but others compile into some intermediate language like LLVM[12] or assembly code that needs further processing to allow for a degree of platform independence. These need to be linked to some kind of assembler in order for the target code to be run (LLVM has a built in assembler called `llvm-as`).

Chapter 4

Project Plan

In this project I intended to complete the following tasks as a minimum:

1. Complete Odersky's Scala course and familiarise myself with Scala development
2. Create an implementation of Finite State Automata
3. Implement Thompson construction Automata translation
4. Write input language specification
5. Implement input lexing and tokenisation
6. Implement creation of lexer from input file

I am confident that I have now completed these tasks and the program is in a state that I am satisfied with. as an extension, I intended to also produce a parser generator, which would require the following tasks:

1.

Completed tasks So far, I have completed approximately 3/4 of the Scala course, with an aim to finish within a week. I have also created a high-level skeleton for the lexer generator that can be implemented.

4.1 Tasks to be completed

Once I have completed Oderskys Scala course, I will need to write my git hooks and unit tests to ensure that code that I commit will compile, run, and pass tests, providing some guarantee as to whether the generator will produce a correct compiler, at least for most cases.

First, I will need to design the lexer generator and its components, starting with the context free grammar that will act as the meta-language. I will then need to create a method of building Deterministic Finite state Automata from the input language.

Once this is completed, I will write some unit tests to test against some language and input with a known result, and then I will build the components.

4.2 Task interdependence

Below is the original table of planned tasks and their dependencies and predicted finish times for the main project deliverables and also possible extension tasks. Each of the core tasks has been completed.

Task no.	Task	Dependencies	Expected time (Days)
1	Write git hooks	N/A	1
2	Design Lexer meta-language	1	3
3	Design Lexer generator structure	2	2
4	Design output lexer structure	3	2
5	Design DFA implementation	2,3	2
6	Write DFA implementation	5	12
7	Finish Lexer generator implementation	6	5
8	Test generator	6,7	5

Table 4.1: Task interdependencies within mandatory tasks

The following table details additional tasks that were planned but not carried out. These are described in further detail in section 7.2

Task no.	Task	Dependencies	Expected time (Days)
9	Create Fuzz testing module	8	7
10	Fuzz testing of generator	8	3
11	Integrate Fuzz testing module into generator pipeline	9	2
12	Design Parser meta-language	2	3
13	Design Parser generator	7,12	3
14	Implement Parser generator	13	14
15	Test Parser generator	14	3
16	Fuzz testing of Parser generator	9,15	2
17	Implement Fuzz testing into Parser generator pipeline	9,16	3
18	Integrate Lexer and Parser generator pipelines	7,14	1

Table 4.2: Interdependencies for extension tasks

Chapter 5

Implementation

The ScalaLex application can be divided into two sections:

1. Processing of Regular Expressions into DFA
2. Processing of the Input language into rule/action parings

Each section has its own tasks and challenges, however both sections can be categorized as specialised compilers.

5.1 Processing of Regular Expressions

Regular expressions have equivalent expressive power to a Finite State Automata, meaning that semantically, they are interchangeable, but, there are several implementation differences between a regex and an FSA. The first is that a regex can be used to find partial matches, i.e. a regex can start and end anywhere within a given input string. This means that there can be multiple different matches for one input string, for example, the regex *baa* would match at 5 different positions in the input string "baabaabaabaabaa". In contrast, an FSA can only produce complete matches on its input string. This presents an issue for lexical analysis which can be solved by feeding in the input string with increasing length, and taking the longest match from the start of the input, or the *longest prefix match*.

5.1.1 Pipeline Overview

Regular Expressions are translated into a DFA by using a pipeline of steps:

1. Lexing of symbols
2. Pre-processing of compound/special operators
3. Creating Thompson constructions
4. Epsilon closure of final automaton
5. Cleaning up redundant states

5.1.2 Lexing of Regular Expressions

In order to support pre-processing, and to catch errors in the regular expression, it is first tokenised by encoding symbols with necessary information. Tokenisation takes place in the `makeSymbol(s,a)` method, that take as parameters a list of characters `s` (from the regex) and a list of `RegexToken a` (the accumulating parameter (see Programming techniques - Recursive Optimisation)).

Regular Expression Tokens Tokens for regular expressions are implemented using case classes that inherit from the class `RegexToken`. Each `RegexToken` contains a character that identifies the Token's corresponding symbol. Types of token are as follows:

- Input
- Operator
- BracketExpression
- CharSet
- QuoteSeq
- Quantifier

Input Input tokens are tokens that are not included in the set of operator symbols that is defined in the `RegexParser` object. The set of operators is as follows:
$operators = \{?, ", -, ', |, *, +, \langle \text{ backspace } \rangle, (,), \backslash, [,], \{, \}, /, \$\}$
$\langle$ backspace $\rangle$ refers to Unicode character `'\u0008'`. The symbol for input tokens is just the input symbol they represent, e.g. `Input(s).symbol == s`.

Operator Operator tokens are either single character operators such as `*`, `+` and `?`, or brackets before they have been through pre-processing. Operator tokens have a Boolean flag that indicates whether that operator was escaped using a backslash ('\') symbol. If an operator is escaped, then it is treated as an ordinary input token. The symbol for operators is their corresponding regex symbol, e.g. `Operator(*,false).symbol == *`.

BracketExpression a `BracketExpression` is a token containing a list of regular expressions between two bracket operators. These are created by the pre-processing function `makeExpression(s)` where `s` is a list of `RegexToken`s. BracketExpressions are processed recursively. BracketExpressions have the symbol '(.

CharSet a CharSet is created from an expression between two braces, e.g. `[0-9]`, which represents the input 0|1|2|3|4|5|6|7|8|9. CharSets are treated as a single compound input symbol (see fig 5.7). Character sets can be inverted by prepending the symbol ^to the set like so: `[^0-9]`. CharSets have the symbol '[.

QuoteSeq a QuoteSeq is a list of characters that have been escaped using a pair of quotations (`"`), for example, the regex `"a+b*c"` is equivalent to `a\+b\*c`. They are treated as a sequence of input symbols in the Thompson Construction phase. the `QuoteSeq` case class contains a list of characters that represents the characters within the quotation marks. QuoteSeqs use the symbol '`"`'.

Quantifier A Quantifier is a special compound operator of the form $\{x, y\}$ that allows you to specify limits to the number of matches for a symbol. The first number x specifies a minimum number of matches, and the second number y specifies the maximum number of matches, for example, the regex `a{1,3}` matches `a`, `aa`, and `aaa` but not `aaaa`. y can be omitted for different effects. $\{x\}$ specifies an exact number of matches, i.e. match only x times, whereas $\{x, \}$ specifies a variant of the kleene star, i.e. match x or more times. The `Quantifier` case class contains a reference to the previous symbol that was created, as well as a minimum and maximum value. Quantifiers are produced by `makeSymbol` but the reference to the previous symbol needs to be updated during the pre-processing of brackets. Quantifiers are compiled away using the function `quantExpand(s)`. Quantifiers use the symbol '`{`'.

Escaping operators As usual, it is possible to escape an operator, i.e. treat the operator symbol as a regular symbol by using a backslash. `makeSymbol()` checks for '\' characters, and if it finds one, it sets the 'escaped' flag for the next operator to be true.

5.1.3 Thompson Constructions

To translate a regular expression, the regex engine first converts the regular expression into a non-deterministic Finite State Automata, or NFA. This is done using Thompson's constructions, as well as two modified versions of the star translation, plus (+) and the lazy operator (?). Once all of the regular expression symbols have been processed into a single NFA, the final state of the NFA is marked as being an accepting state.

Input symbols For a standard input symbol, that is, any symbol, or set of symbols not treated as an operator, the regex engine will create an automata with two states with a transition for the input symbol x from state $q0$ to state $q1$, as follows:

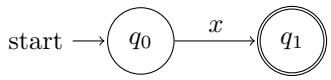

Figure 5.1: Basic Thompson construction

Concatenation Concatenation is converted into a transition from state n in the first NFA p to state 0 in the second NFA q. Additionally, concatenations can be inserted by using the look-ahead operator '/', although this is unnecessary because concatenations between symbols are inserted during pre-processing and partial matches are not supported, look-ahead is logically equivalent to a concatenation.

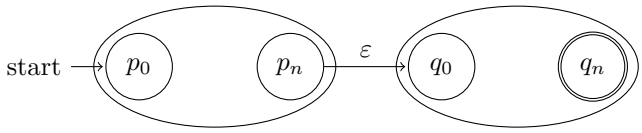

Figure 5.2: Thompson construction for concatenation

Star $*$ The Kleene star, representing zero or more of some NFA q is translated by creating two new states, s_x and s_y, and by adding an epsilon transition from state s_x to state s_y, from state s_x to state q_0 of the NFA, from state q_n to state s_x, and from state q_n to state s_y. State s_y becomes the final state of the new automata.

Because the Regex engine eventually compiles these automata into DFAs using epsilon closure, the epsilon transition from state q_n to state x could also be changed to a transition from state y to state x with no change in semantics. This is because s_x is connected to q_0 using no input and q_n is connected to s_y using no input, so they will become part of the same DFA state after epsilon closure.

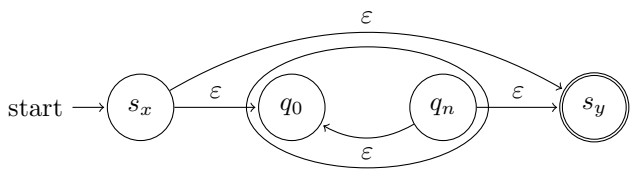

Figure 5.3: Thompson construction for Kleene star

Union | The Union of two NFAs is constructed by creating two new states, s_x and s_y, and by adding an epsilon transition from state s_x to state q_0 of both NFAs, and from state q_n of both NFAs to state s_y. State s_y becomes the final state of the new automata.

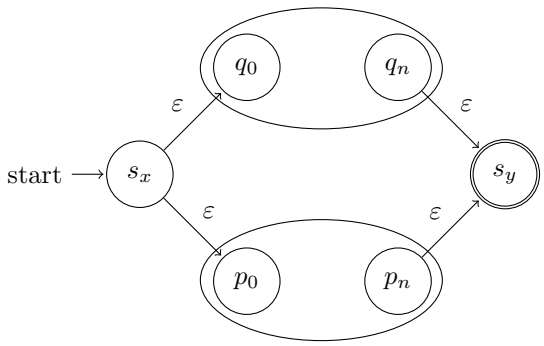

Figure 5.4: Thompson construction for union

Plus $+$ To translate the $+$ symbol into an NFA, the Kleene Star construction can be modified by omitting the transition from state s_x to state s_y, meaning that the NFA must be traversed at least once.

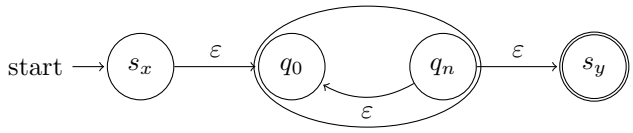

Figure 5.5: Modified Thompson construction for Plus

Lazy ? To translate the ? symbol, the Kleene Star construction can be modified by removing the transition from state q_n to state x, preventing repeated matching.

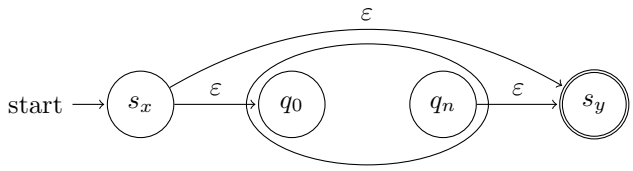

Figure 5.6: Modified Thompson construction for Lazy value

5.1.4 Special case operators

The following features of regular expressions are supported by converting them into some combination of the previous NFA constructions using a pre-processing pipeline. The implemented pre-processing pipeline is as follows:

1. Processing Brackets
2. Processing Quantifiers
3. Processing Character Sets
4. Processing Concatenations

Brackets Bracketed expressions are treated as an inner NFA and processed independently from the main expression using a recursive call. If using any of the other pre-processing functions, brackets must be converted first.

Quantifiers Quantifiers of the form $x\{m, n\}$ are converted into m of the previous input x followed $(n - m)$ of the previous input with lazy operators $(x)?$ during pre-processing, e.g. $x\{1, 3\} = (x)(x)?(x)?$.

Character Sets Character sets are treated as a single input token. The Regex engine also supports ranges within the character set by converting a range expression into an explicit list of characters, e.g. $[0 - 9]$ becomes [0123456789]. The state transition class also contains an inverse transition property that is used to process the inversion symbol ^. for a given character set x, the construction is as follows:

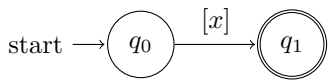

Figure 5.7: Translated input construction for character sets

Concatenations During pre-processing of the regex, concatenations are added between input symbols and after operators for use with the Thompson construction. Concatenations are marked using a unicode backspace character ('\u0008'). If using any of the other pre-processing functions described, concatenations must be inserted last.

5.2 DFA Conversion

Once an NFA has been produced from the original regex, the NFA must be made Deterministic in order to facilitate pattern matching. This is done by using a dynamic programming, graph traversal approach to build new Automata nodes by taking the epsilon closure of each node.

Epsilon Closure Taking the Epsilon closure of a node means:

> Starting from this state, which states are traversible using only ε transitions?

Each unique set of states derived from Epsilon traversal then becomes a state in the DFA.

DFA Algorithm The algorithm used by ScalaLex to convert from an NFA into a DFA uses two phases:

1. Build a new DFA by taking epsilon closures of each state on the NFA, keeping track of how each DFA state is linked by non-epsilon transitions
2. Remove "dead end" states, i.e. states which cannot be traversed in a valid solution to the DFA.

This second phase effectively prunes off any unwanted states that are redundant to the representation.

Epsilon Closure Algorithm

5.3 Lexing of Input language

The input file for ScalaLex follows the same specification as Lex, with the one exception being that code sections in ScalaLex are written in Scala. The input file is split into three sections, which are separated by a line containing the characters `%%`. The first section contains preamble code, definitions, lexing options, and lexing state declarations. The second section contains lexing rules, and the third section contains free-form code. Comments may be present in any section, but are ignored by the compiler.

Comments Comments are any statements enclosed between a `/*` and `*/`. comments can be placed anywhere in the file.

Definitions Definitions are comprised of:

⟨Definition⟩::= {⟨Name⟩} ⟨Pattern⟩

where ⟨ Name ⟩ is a plain text identifier label. Whenever the ScalaLex compiler sees {⟨ Name ⟩} in a rule, the compiler will automatically substitute the name for its associated regex in the final version of the rule.

Declarations Declarations are used to define preamble code and lexing states, as well as enabling optional features in the output lexer. Definitions are of the form:

⟨Declaration⟩::=`%option` ⟨Option⟩
| ⟨States⟩
| ⟨Scala⟩
| %{⟨Scala⟩}%

Options Options are statements that allow you to declare option flags for the output lexer. This is a potential extension to ScalaLex because currently there are no options available to the output so this type of statement has no effect on the result program. Option statements are of the form `\%option` ⟨ Option ⟩ which will enable the functionality of ⟨ Option ⟩ in the output file.

Lexing States Declaration statements can define a set of lexing states for the output lexer, which can either be inclusive or exclusive. Lexing states can be prepended to a

⟨States⟩::=`%x` ⟨StateList⟩
| `%s` ⟨StateList⟩

⟨StateList⟩::=⟨State⟩ ⟨StateList⟩
| ⟨State⟩

Rules Rules are pairs of regular expressions and a Scala expression that form the core of the output program. When the output lexer scans an input, it checks it against the rules for the longest matching rule.

⟨Rules⟩::=ε
| ⟨Rule⟩
⟨Rules⟩

⟨Rule⟩::=⟨RuleStart⟩ ⟨Scala⟩
| ⟨RuleStart⟩ %{⟨Scala⟩}%

⟨RuleStart⟩::=<⟨StartConditions⟩> ⟨Pattern⟩
| ⟨Pattern⟩

⟨StartConditions⟩::=⟨State⟩

Code Arbitrary Scala code (⟨ Scala ⟩) can be inserted at several positions within the input file. in the ⟨ Defs ⟩ section, Scala code can be inserted by indenting or by using a scala code block. Scala code sections are required after a pattern and a tab space in each ⟨ Rule ⟩. Note that code sections can be empty, resulting only in the matched text being consumed. The final section of the input file is reserved for any additional code you may wish to include at the bottom of the program, such as post-processing or routines to call in your rules.

5.4 Creation of Output program

The final lexer is produced by the factory object `LexerFactory`. The output program is built line by line using a string builder. I used a string builder because the program does not need to be modified at runtime and each component is independent. The output file structure is as follows:

1. Neccesary import statements
2. Code from the ⟨ Defs ⟩ section
3. State variables
4. In-built procedures
5. `yylex()` function
6. rule code
7. supplementary routines

Some procedures, such as yylex() are pre-defined within `LexerFactory`, and are unaffected by the structure of the input file, they will always be included in the final program.

5.5 Programming techniques

The following paradigms and techniques were utilised in order to produce the final program.

5.5.1 Functional Programming

Functional programming is a programming paradigm based on the Lambda Calculus[6] that is founded on the principle that all elements of a programming language can be represented by a function. Scala incorporates most of the aspects of functional programming but remains a hybrid language because of its support for objects and the fact that Scala supports side-effect functions, i.e. functions that do not return a value, for I/O functions such as printing lines to the console.

ScalaLex conforms loosely to the functional programming paradigm, and takes advantage of many functional programming techniques such as pattern matching, map and filter functions.

Pattern matching A major feature of most functional languages is the ability to pattern match against some input. In most languages this is incorporated through switch statements. Scala uses 'patterns'[15] to implement switching behaviour. ScalaLex utilises patterns in many parts of the application.

Anonymous functions One of the most important features of a functional language is the ability to create anonymous functions, that is, functions defined without a specific method name. These anonymous or lambda functions are derived from the Lambda calculus and allow for the definition and use of a function in the same without binding it to an identifier. This effectively allows for the use of a function in the place of a literal. This is part of the idea that a function can be used in the place of any other object[15], and is often used in conjunction with higher order functions, i.e. functions that use a function as a parameter. Anonymous functions are used in ScalaLex in a conventional function style in conjunction with map and filter functions to create and search collections such as components of the processed regex or processed input file.

5.5.2 Recursive optimisation

Many components of ScalaLex use recursive iteration. Given the size and complexity of the inputs of the application, some effort has been made to ensure that the recursive components will terminate in the most efficient way possible by utilising compiler optimizations to reduce the size of the call stack.

Accumulating parameters By adding an extra parameter to a recursive function to store intermediate results, a recursive method can be optimized into a while loop. This is because by storing all the information needed for a recursive call in the parameters each recursive call does not need to return any information to the previous one. This means that you can execute the recursive method using only one stack frame.

Tail-recursive functions Tail recursive functions are recursive functions that only use a recursive call at the end. This also allows for the function to be optimized into a while loop which eliminates the need for additional stack frames within the function and helps prevent stack overflow exceptions due to the limits imposed by the JVM. This complements the accumulating parameter technique to create very efficient recursive functions.

Wherever possible, I have made use of the Scala `@tailrec` annotation in my recursive functions. This annotation is used to mark tail recursive functions that can be optimised into a while loop.

5.5.3 Object Oriented Programming

In the creation of FSAs, I have used an object oriented approach, where each FSA is an object that contains a collection of state objects. Each of these objects have accesor and mutator methods that allow for the construction of the DFA through the Thompson constructions, and through Epsilon closure.

Abstract classes I have used abstract classes to reduce duplication of code in the State classes by defining one for FSAs and one for states that is used by both the NFA and DFA classes. Abstract classes help to improve the extensibility of ScalaLex by reducing duplication in class methods and creating a coherent class structure.

Type parameters In order to incorporate all of the common properties into the abstract classes, I have used type parameters and type variables to create methods that take generic types. Type parameters are used in conjunction with abstract classes to generalise methods that use types that inherit from abstract classes.

Case classes In the lexing process for the input file and the Regexes case classes are used to represent each token. Case classes are a Scala feature that is used to create lightweight classes to allow for more finely tuned pattern matching. This is a perfect fit for the task of lexing because it allows you to construct objects that encapsulate the lexical information for a single token. Case classes differ from normal classes in that they can be created without using the 'new' keyword, and the `copy()` method can be used to create a shallow copy and parameters can be modified as desired.

5.5.4 Serialization

In order to utilise the DFA in the output program, the DFA objects which are created by ScalaLex at runtime must be stored somehow for use in the final program. There are three ways to approach this problem:

1. Run the output program as part of the ScalaLex pipeline
2. Include the Regex parser with the final program
3. Store the DFA objects during the ScalaLex runtime and include them with the final program.

ScalaLex uses option 3. I have chosen to do this through serialization because this allows me to reduce the startup cost of the output program and maintain separation between different components in ScalaLex. Serialization still requires the custom state library to be included in the final program by packaging it into a jar and adding it to the JVM classpath through a build tool (in this case sbt). In future versions, this will most likely be replaced by a library

hosted in a repository such as Maven. ScalaLex serializes a list of pairs of type `List[(DFA,String)]` which encodes the DFAs as well as the regex that produced each DFA. The DFAs are checked during `yylex()` for matches, and if matches are found, then the longest prefix match is taken. For each execution of `yylex()`, the longest of these matches is selected and the rule associated with the chosen DFA is executed by calling `doRule()` with the chosen regex.

5.6 Limitations of ScalaLex

ScalaLex does have some limitations due to its limited feature set. ScalaLex supports most of the basic features of lex and flex, but there are some features that are not supported, and there are some features that are only partially supported. Planned features are detailed in section 7.1.

5.6.1 Issues encountered during development

The following is a list of issues that have been fixed throughout the project, as well as known issues with the current version of the application that should be addressed if the project is to be continued.

Reading rules with no action Rules that do not implement an action, i.e. rules that simply remove matched text from the input cause an error in the ScalaLex input lexer that registers the empty rule and the one after it to appear as one rule merged together, for example

```
delete me
abc    println("abc")
```

becomes

```
delete meabc    println("abc")
```

Deserialization Currently, the final program will not run without adding the `DFA` class to the JVM classpath, meaning that the user must create a jar file for scalalex and include it into there system classpath. In future this will be fixed by adding build

Manual setup As well as adding ScalaLex to the JVM classpath, additional setup is required from the user to work the final lexer. The user needs to resolve the dependencies packaged into the output program, as well as any additional packages added to the program manually by placing the source file into a sbt or IntelliJ project and editing the build file.

Chapter 6

Other applications

6.1 LaTeXmarkup

A popular tool for writing technical documents is the LaTeXtext editor. LaTeXuses a markup language to define the logical structure of a document, and then uses a compiler to convert the markup into a PDF format for publishing. It would be possible to create a LaTeXcompliant lexer using ScalaLex by writing a LaTeXgrammar as the input rules. It would also be possible to create a LaTeX compiler or transpiler[9] that would convert from a hybridised syntax for a library into pure LaTeX or from one form of LaTeX into another.

6.2 Emails

One potential application for this tool would be in producing an email processing system that would look for keywords in the email to determine the type and nature of the email, for example, creating automated spam filtering systems, tagging emails based on subject matter, and other natural language processing systems. This would require some thought about how to handle matches that fit into multiple categories etc. The most obvious application for this would be a rudimentary spam filtering system that is designed to prevent harmful emails from being opened whilst catering to the user's preferred level of marketing email[4].

6.3 Text adventure games

Another application would be producing a text-based adventure game, such as the classic game series Zork[13]. This would involve matching keywords to determine commands, and printing an output based on the context. A main function would need to be used to create an I/O processing loop that will continuously ask the user for commands.

Chapter 7

Future work

Although the ScalaLex project is in a near complete state, there are still improvements and extensions to be made to the application to improve both its user friendliness and its utility. This is because ScalaLex proved to be a more complex task than I first anticipated and other time constraints limited the amount of time I was able to spend on the project, meaning that although I am confident in the quality of the final work, it is still a prototype, and there is much room for improvements and additional features to be added.

7.1 Planned features

The following are features that are implemented by lex and flex but are not currently implemented by ScalaLex. These features were planned to be implemented but were disabled due to lack of time.

Lookahead The lookahead, or trailing context operator `/` is used to match text without consuming it, for example the pattern `r/s` would match `r` followed by `s`, but only `r` would be consumed from the input string, allowing `s` to be matched against another pattern. Similar behaviour can be achieved by utilizing the `unput()` function, which allows you to add characters to the input string.

Start matching Start match is a special case of the look-ahead operator, where the string `"$abc"` is equivalent to `"`ε`/abc"` [14]. In ScalaLex, the `$` operator is used to create regexes that will only match if none of the input string has been consumed. Start match and end match are facilitated via a boolean flag property in the FSA class.

End matching End matching is another special case of the look-ahead operator that only matches if all of the input has been consumed. The end match operator `abc^` is equivalent to `abc/`ε

At the time of writing, the lookahead (`/`), start match (`$`) and end match (`^`) symbols are lexed by the regex parser but throw a NotImplementedException.

Reject function flex and lex include a function call `REJECT;` that instantly terminates the lexer with an input rejection that can be called from the rules. ScalaLex should implement similar functionality in future to refine pattern matching functionality.

Program entry point Currently ScalaLex will produce a near complete lexer from the input, however some components still need to be implemented manually by the user either as supplementary code from the input file or by editing the resulting source file. The primary missing component is a main function that can act as an entry point to the program that will execute and call `yylex()` on the input string to the lexer. This can be included in the last section of the input file and it will be appended to the resultant lexer class, but it would be desirable to add a default main function that can be enabled using an option parameter in the input file, for example `%option main,`. The primary purpose of the main function in most cases should be to call `yylex()` on the input text until either a match has failed or the text has been consumed.

Additional functionality from lex Additional work can be done to achieve the functionality of Lex/Flex by adding features that are implemented by Lex and Flex, such as character classes. Character classes are pre-defined regex patterns that represent multiple characters[16], for example, in flex the class `[:alnum:]` represents any alphanumeric character, and is equivalent to the pattern `[0-9a-zA-Z]`. By implementing these features as closely as possible to lex/flex, it would be easier to add compatibility between the programs.

As part of this compatibility, it would be useful to write a script that will convert Lex files into ScalaLex, and allow for backwards compatibility between (F)Lex and ScalaLex. Additionally, a script could be written to convert ScalaLex files into Lex/Flex.

Organising Libraries In order to make publishing ScalaLex easier, it would be worth separating out the project into distinct modules that ScalaLex uses, as the States/FSA module could be made independent from the rest of the application, allowing it to be used in other contexts and the Regex translation module is also independent from the main program but not the State module.

7.2 Other planned projects

After completing ScalaLex I also planned to produce several other modules that would provide extra functionality.

Fuzz testing For more rigorous testing methods, I intended to utilise fuzz testing[11] in order to improve the robustness of the generator and add confidence to the produced lexers. Due to the complex nature of the input specification for a lexer, there is lots of room for errors to be made, meaning that it is difficult to test edge cases without writing a very large number of tests, which requires lots of time and maintenance. When producing tests for a lexer generator, this task becomes even harder because each run is potentially producing a different program with different but equally complex input characteristics.

Parser generator As an addition to ScalaLex, it would be logical to create an allegorical program to YACC, i.e. a parser generator that produces a Scala program. Parser generators take a set of context-free grammars as input and produce a parser that corresponds to the grammar. Like lex and YACC, this could be linked to ScalaLex to produce a pipeline that will combine the different modules[2].

Chapter 8

Conclusions and Reflection

Most of the challenge and complexity of this project comes from the involvement of meta-programming and of handling unbounded user input. These two properties significantly increase the possibility of mistakes and problems appearing which makes the software engineering practices used even more valuable.

Overall I would consider this project to be a success, because most of the primary aims have been met. A tool has been produced that significantly reduces the amount of work needed to create a lexer in the Scala language. I have learned much about the process of producing lexers, software design, automation, testing and how to create tools for use by other developers. I have put into practice many of the skills I have learned throughout my course such as functional programming, software engineering and compiler design.

If I were to attempt this project or a similar project again, I would attempt to produce tools that integrate with existing libraries, for example in the case of ScalaLex, a version 2.0 would add compatibility with an existing FSA library to make it easier to produce the output program, as many issues with serialization and other aspects can be solved by using an existing library. If I was producing ScalaLex as a product, this is the approach I would take, but since this project was mainly about researching and testing my abilities, it made more sense to produce the FSA library myself as this would allow me to gain deeper insight into what was involved in producing these FSA.

Bibliography

[1] Alfred V Aho, Ravi Sethi, and Jeffrey D Ullman. "Compilers, principles, techniques". In: *Addison wesley* 7.8 (1986), p. 9.

[2] Doug Brown, John Levine, and Tony Mason. *Lex & yacc*. " O'Reilly Media, Inc.", 1992.

[3] Scott Chacon and Ben Straub. *Pro git*. Apress, 2014.

[4] Rui Chen et al. "An investigation of email processing from a risky decision making perspective". In: *Decision Support Systems* 52.1 (2011), pp. 73–81.

[5] Noam Chomsky. "Three models for the description of language". In: *IRE Transactions on information theory* 2.3 (1956), pp. 113–124.

[6] Alonzo Church. "An unsolvable problem of elementary number theory". In: *American journal of mathematics* 58.2 (1936), pp. 345–363.

[7] *Comparison of parser generators*. URL: https://en.wikipedia.org/wiki/Comparison_of_parser_generators.

[8] François-Nicola Demers and Jacques Malenfant. "Reflection in logic, functional and object-oriented programming: a short comparative study". In: *Proceedings of the IJCAI*. Vol. 95. 1995, pp. 29–38.

[9] John J Donovan. *Systems programming*. McGraw-Hill, Inc., 1972.

[10] Cristian González Garca et al. "Swift vs. objective-c: A new programming language". In: *IJIMAI* 3.3 (2015), pp. 74–81.

[11] Patrice Godefroid, Michael Y Levin, David A Molnar, et al. "Automated Whitebox Fuzz Testing." In: *NDSS*. Vol. 8. 2008, pp. 151–166.

[12] Chris Lattner and Vikram Adve. "LLVM: A compilation framework for lifelong program analysis & transformation". In: *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. IEEE. 2004, pp. 75–86.

[13] P David Lebling, Marc S Blank, and Timothy A Anderson. "Special feature zork: a computerized fantasy simulation game". In: *Computer* 4 (1979), pp. 51–59.

[14] Michael E Lesk and Eric Schmidt. *Lex: A lexical analyzer generator*. 1975. URL: http://dinosaur.compilertools.net/lex/index.html.

[15] Martin Odersky et al. *An overview of the Scala programming language*. Tech. rep. 2004.

[16] Vern Paxson. *flex linux man pages*. URL: http://dinosaur.compilertools.net/flex/manpage.html.

[17] Reinhard Wilhelm, Dieter Maurer, and Stephen S Wilson. *Compiler design*. Springer, 1995.

[18] Lihao Yi. *FastParse 2.2.2*. URL: http://www.lihaoyi.com/fastparse/.

Appendix A

ScalaLex Readme

This readme is included with ScalaLex and is intended to provide a user-level description of the functions and limitations of ScalaLex.

A.1 ScalaLex

The following readme file is supplied with ScalaLex and is available on Github[1]

This tool is designed to implement a modified version of the LEX[2]/FLEX[3] specification. This program implements all of the standard functions of lex or flex, but the output program is written in scala.

ScalaLex uses a Regex Compiler that produces DFA equivalents of each of the regex rules in the output program. Each Regex is matched using longest prefix match, and in the case of two or more matches of the same length, the first matched rule is used.

A.1.1 Requirements

This tool was built using sbt and requires Java 8 to run.

A.1.2 Instructions for use

To use this program, you need to first package it into a jar file by running the command `sbt package` (or just `package` if inside sbt), which should create a jar file in the /scalaLex/target/scala-2.12 directory.

This file needs to be added to your system classpath to allow the output program to run. To do this you need to add the jar file to your build file using IntelliJ or sbt (for sbt see https://www.scala-sbt.org/1.x/docs/Classpaths.html), run the command: `set fullClasspath in Compile += Attributed.blank(file(".../ScalaLex/target/sca 2.12/ScalaLex.jar"))`

You can check your classpath by using the command `show compile:fullClasspath`

The arguments for ScalaLex are as follows:

[1] <https://github.com/stephen-bracken/ScalaLex>
[2] <http://dinosaur.compilertools.net/lex/>
[3] <http://dinosaur.compilertools.net/flex/manpage.html>

```
- -i : the input file to read defninitions from
- -o : the name of the output source file to be created
```

e.g. running `java -jar scalaLex -i input.txt -o output` will read definitions from the source file input.txt (an example has been provided for you) and output to the file /output/output.scala. Note that ScalaLex will always produce a file named dfa which is **always necessary** for running the output program.

sbt build file

To resolve dependencies in the output program, you will need to use a build tool such as sbt or IntelliJ, and add the following dependences to the output program in a build file (build.sbt for sbt):

```
 libraryDependencies ++= Seq( "org.slf4j"%"slf4j-api"%"1.7.25",
//simple logging facade for Java "ch.qos.logback"%"logback-classic"%"1.2.3",
//logback "ch.qos.logback"%"logback-core"%"1.2.3", "com.typesafe.scala-
logging" %% "scala-logging" % "3.9.2", //scala-logging )
```

A.1.3 Input Language

The top level syntax for an input file is:

```
{defs}
%%
{rules}
%%
{user subroutines}
```

the subroutines section can be ommitted, therefore the minimum input specification is

“‘

```
%%
```

“‘

which will produce a program that will simply output any input given.

Definitions, Options and States

the first section {defs} may contain any of the following expression types:

- `%option` Lexing options for the generated lexer
- `%x INSTRING INCOMMENT` lexing states to enable/disable rules that are either exclusive (%x) or inclusive (%s)
- `{name} regex` associates a name with a regex. The name can be called in the rules section e.g. `{number} [0-9]+` can be called using `{number}` and the call will be replaced with `[0-9]+` in the output regex

- `  println("example code");` indented lines will automatically be treated as code that will be executed when the lexer is initialised. Code in this section is placed before definition of the lexer class so any code here aside from imports should be wrapped in a class or object to prevent compiler errors.
- `/* comment */` /* and */ can be used to declare a comment.

Rules

a rule is of the form `<StartCondition> Regex action`

start conditions may be omitted but there must always be a regex that ends with a tab space. Actions are comprised of arbitrary scala code. If the start condition is omitted, it is assumed to be `<INITIAL>`, which is the default state of the lexer if no states are added in the defs section. Switching lexing states can be done in actions.

example

```
 hello println("hello")
```

this rule uses the regex `hello` which matches the word "hello", and as it's action, it prints "hello" to the console.

Subroutines

the user subroutines section consists of a continuous code block that can be filled with any additional functions that you want to reference using either rules or the initial code. These will be included in the lexer class at the bottom.

A.1.4 Built-in methods

the following methods are built into the output file to allow for utility functions and define a standard pipeline for lexing:

- `readFile()` reads the file from the supplied location and writes it into the input string.
- `input()` gets the next character in the input stream
- `output()` writes a character to the output stream
- `unput()` writes a character back onto the front of the input stream
- `yylex()` runs each regex on the input string and consumes the longest matched string. If an action is associated with the regex, then the action method is called.
- `doRule(r)` is used by yylex to call code actions.

A.2 Regular expression support

A.2.1 Regex Operators

- `{x,y}` matches the previous symbol a minimum of x times and a maximum of y times. the max parameter is optional
  - `a{1,3}` matches 1 to 3 instances of a
- `[]` creates a set of union operations between characters. Includes support for character ranges, e.g.
  - `[0-9]` is equivalent to (0|1|2|3|4|5|6|7|8|9)
  - `^` inverts the character set, matching any character **except** the characters in the set, e.g.
  - `[^0-9]` matches any character except `[0-9]`
- `()` standard bracketing
- `\` escape character
- `*` kleene star, i.e. 0 or more of previous symbol
- `+` like kleene star, but with 1 or more, e.g. a+ is equivalent to aa*
- `|` union/or operation, i.e. previous symbol or next symbol
- `"` quote sequences are used to escape all operators in a sequence, e.g. "xyz++" is equivalent to the input sequence xyz++
- `/` lookahead operator - when translated this becomes a concatenation, because the automata always matches all of the input sequence*

note that it is possible to get partial matches from the getMatches and longestPrefixMatch functions, but the lookahead functionality remains the same.

Special operators

the following operators are used during the processing of an input string, and will be escaped.

- `\u0008` is a backspace character that is used to represent concatenations
- whitespace is used for parsing rules and definitions, so to use whitespace in your regex, it is highly reccomended to parenthesise it using `[ ]`

Planned features

- `/` trailing context - `a/b` matches ab but returns b to the input
- `$` special case lookahead - checks that the entire input has been consumed. Equivalent to `x/\n`
- `^` lookbehind - checks that the beginning of the string was matched
- `.` matches any character

- `{x,}` matching x or more characters
- multiple start state matching, e.g. `<INSTRING,INCOMMENT>x`
- `«EOF»` end of file
- character classes, e.g. `[:alnum:]` = `[]`
- `REJECT;` rejects this regex and moves onto the next best match.

Appendix B

Tools and Abbreviations

[H] Below is a list of abbreviations of tools and techniques with a brief description of the tool and the main website associated with it.

#	Abbreviation	Full name	Description	Website
1	GPL	General-Purpose Language	A widely applicable programming language designed for use on any programming problem.	
2	DSL	Domain Specific Language	A language specialised to a particular application domain, such as HTML	
3	DFA	Deterministic Finite state Automata	A finite state machine with a single defined path for each valid input string.	

4	LL	Left to right, Left recursive	A parsing strategy where the input is read left to right (i.e. first to last), and rules are resolved to accommo-date left-recursion	
5	LALR	Look Ahead Left-Right	A parsing strategy which uses a look-ahead method on the input string, reading left to right, and accom-modating right-recursion	

6	RE	Regular Expression	A language capable of expressing lexical tokens that are valid for a given program. Used in pattern matching, a Regular Expression is conceptually isomorphic to a DFA and is often used to define lexer specifications and implementations.	
7	CFG	Context-Free Grammar	A language capable of expressing program structure in the form of an Abstract Syntax Tree. This is the type of language used for Parsing.	
8	EBNF	Extended Backus-Naur form	A family of metasyntax notations expressing a CFG. ISO/IEC 14977	https://standards.iso.org/ittf/PubliclyAvailableStandards/s026153_ISO_IEC_14977_1996(E).zip

9	Lex	Lex A lexical Analyzer Generator	Tool for writing programs whose control flow is directed by instances of regular expressions in the input stream.	http: //dinosaur. compilertools. net/lex/ index.html
10	FLEX	Flex, A fast scanner generator	Flex forms pairs of regular expressions and C code known as rules. Flex outputs a C file.	http: //dinosaur. compilertools. net/flex/ index.html
11	Re2c	Regular expression to C	A lexer generator that produces C and C++ code	https: //re2c.org/
12	ANTLR	ANother Tool for Language Recognition	A parser generator that produces LL parsers	https: //www. antlr.org/
13	GOLD		A lexer and parser generator that separates language syntax parsing from language construc- tion.	http:// goldparser. org/

14	YACC	Yet Another Compiler Compiler	A ubiquitous parser generator that produces LL and LALR parsers	http: //dinosaur. compilertools. net/yacc/ index.html
15	sbt	Scala Build Tool	An interactive build tool for Scala with support for scripts and detailed project configuration	https: //www. scala-sbt. org/

Table B.1: a list of abbreviations of tools and techniques with a brief description of the tool and the main website associated with it.

Appendix C

Language Specification

Below is the full input language specification for ScalaLex:
note: ⟨Scala⟩ corresponds to Scala syntax [15]

ScalaLex Sections

⟨L⟩::=⟨Defs⟩
%%
⟨Rules⟩
| ⟨Defs⟩
%%
⟨Rules⟩
%%
⟨Scala⟩

⟨Defs⟩::=ε
| ⟨Definition⟩
⟨Defs⟩
| ⟨Declaration⟩
⟨Defs⟩

ScalaLex Elements

⟨Definition⟩::= {⟨Name⟩} ⟨Pattern⟩

⟨Declaration⟩::=`%option` ⟨Option⟩
| ⟨States⟩
| ⟨Scala⟩
| %{⟨Scala⟩}%

⟨States⟩::=`%x` ⟨StateList⟩
| `%s` ⟨StateList⟩

⟨StateList⟩::=⟨State⟩ ⟨StateList⟩
| ⟨State⟩

⟨Rules⟩::=ε
| ⟨Rule⟩
⟨Rules⟩

⟨Rule⟩::=⟨RuleStart⟩ ⟨Scala⟩
| ⟨RuleStart⟩ %{⟨Scala⟩}%

⟨RuleStart⟩::=<⟨StartConditions⟩> ⟨Pattern⟩
| ⟨Pattern⟩

⟨StartConditions⟩::=⟨State⟩

Appendix D

Supervisor meetings

In this section, the notes of meetings with Dr Martin Berger and Dr Ian Mackie are listed from throughout the project.

2nd October 2019 Martin and I discussed the initial research topic area, the structure and outline of the project, what languages I would use, and any software engineering techniques I would use to help automate testing processes throughout the project. We decided that the focus of the project would be the generation of syntax checking tools, and that this would be done in Scala. Martin also suggested that I should use an automated testing pipeline to ensure that any code committed to the projects git repository should be working code.

11th October 2019 We discussed writing the project proposal, tools for automation, existing compiler generation tools, and alternative methods of writing compiler pipelines. Martin suggested that to improve automation and integration of tests, I should write scripts using git hooks, a simple tool that allows you to create bash scripts that run when certain git events occur. Combinator parsing was also discussed, whereby a parser is written that will lex a context free grammar, eliminating the need for a lexer to be prepended to the pipeline at the cost of speed and error handling. As a next step, Martin recommended that I should take a short course (Odersky, n.d.) on Scala techniques.

23rd October 2019 I updated Martin on my progress through the Scala course and the specifics of implementing a Lexer generator were discussed. We decided that in order to fully express regular expressions, a meta-language would need to be created and expressed as a context free grammar. This means that the lexer generator must be implemented as a special-case of a parser.

30th October 2019 Once again, I updated Martin on my course progress and we broadly discussed writing the interim report. Martin suggested that I should focus on the background research and mention any software engineering techniques I am/will be using to test my work.

8th November 2019 Martin and I discussed what form the generator's grammar would take, and the possibility of using the Lex/Flex grammar was first brought up here. The final grammar for

13th December 2019 I discussed good software engineering practices with Martin. ScalaCheck was mentioned here. Automation of tests using a CI pipeline was suggested, as the program is very complex and vulnerable to mistakes We talked about technical aspects of implementing automata, such as how to deal with the complexity of the epsilon closure process. We talked about using a post-processing technique to remove redundant states from the DFA. Finally we discussed the possibility/feasibility of implementing error checking into the automata pipeline which would track which symbols caused errors.

23rd January 2020 Martin and I talked about pre-processing the regex into an abstract syntax tree to make processing the automata easier, as well as being able to detect errors in the regex early, and implementing additional features easier. We discussed the concept of the regex to automaton translator as being a special case of a compiler. Using case classes to implement an abstract syntax tree was suggested. We also considered the possibility of adding support for JFlex.

7th February 2020 Ian and I discussed my progress on the project so far, and Ian gave several suggestions for the final report. Ian suggested building test cases for other forms of input language, such as a DSL or a markup language like XML.

19th February 2020 Ian suggested testing the application on other kinds of input, such as creating a text-based game, or creating automated responses to emails. We discussed adding error handling to the main program. Ian gave me more advice on what to include in my report.

10th March 2020 I briefly discussed the project poster with Ian, and he suggested as an extension task to write a script that will convert a Lex file into a ScalaLex file. Ian advised me to include theoretical aspects in my final report to show how I have applied knowledge gained from my degree.

Appendix E

Original Project Proposal

Below is the original project proposal submitted to Dr Martin Berger.

Lexer Generator

Stephen Bracken 2019

Background

Programming languages are an essential part of computer science, affecting every area, as programming languages allow programmers to create abstractions from instructions for individual pieces of hardware and implementation details that may differ from one user to another allowing us to export programs to many different users with many different environments. By offloading memory management and process control onto the Operating System, programmers can create larger, more complex programs. This is achieved by using a compiler to convert between the programming language and the instruction set for the machine. Modern compiler pipelines first check the syntax of the program to enforce language constraints and then compile and optimise the code.

Project aim and objectives

The aim of this project is to explore different methods of constructing compilers and to build a program that can create syntax checkers (i.e. lexers and parsers) by deriving them from language syntax. There are two methods to do this:

1) Implement a lexer and a parser separately and combine the two into a pipeline
   or
2) create a combinator parser that will check both the validity of program tokens and program syntax.

Goals/Outcomes

The initial goal of this project will be to produce a lexer generator, that is, a program that takes a regular expression as input, and returns a lexer that corresponds to the list of tokens given using the lexer generator.

Due to the combination of OO and functional paradigms, Scala is a good choice of language for this task because it has in-built pattern matching that will be very useful, however I will need to spend some time familiarising myself with the language and its environments.

Extensions

To complete the pipeline, it would be useful to create a parser generator, that takes a context free grammar and outputs a parser. The lexer and parser generators could then be pipelined together. I will need to consider the speed of progress of my lexer to decide whether that is possible.

If possible, I would like to extend my project by adding a testing module to the pipeline that would allow the generated compiler pipeline to be tested using generated programs. Additionally, steps can be taken to improve the speed of the parsers that are generated, as there are several ways to simplify each statement.

Depending on the time of completion of the lexer generator and parser generator, I would also like to compare this with a combinator parser, so creating a combinator parser would be a logical next step. I could then compare the speed of the pipeline with the speed of the combinator parser.

Relevance

This project is very relevant to several areas of my course. To carry out this project I will need to use techniques learned from several modules of the course including:

- pseudo code and logical thinking about the problem from Programming Concepts
- algorithm design and complexity analysis from Program Analysis
- functional programming from Comparative Programming
- compiler design from Compilers.

Time allotment

I have 10 hours of contact time this term and 3 other modules to study for. Next term I will have 2 other modules so there will be slightly more time to focus on the project. Taking these factors into account I will aim to spend roughly 10 hours per week working on this project.

Time available:

Monday: up to 4pm

Tuesday: all day

Wednesday: 11am-6pm

Thursday: after 1pm

Friday: 11am-1pm, 2pm-4pm